

FASE 2 - DATOS Y MODELO BASE

FASE 2 - DATOS Y MODELO BASE.....	1
1. EDA - Análisis Exploratorio de Datos.....	2
1.1. Bases de datos: NASA Turbofan (CMAPSS).....	2
2. Calidad de Datos.....	3
2.1. Los Datos del Dataset.....	3
2.2. Datos innecesarios.....	3
3. Feature Engineering.....	4
3.1. Cálculo del RUL en entrenamiento.....	4
4. Pipeline de datos.....	5
4.1. Ingesta.....	5
4.2. Tratado.....	5
4.3. Almacenamiento.....	5
5. Diagrama de flujo.....	6
5.1. main.py.....	6
5.2. PIPELINE.....	6
5.3. procesar_varios_archivos.....	7
5.4. assign_column_names.....	7
5.5. sensores_irrelevantes.....	7
5.6. calcular_rul.....	7
5.7. spark.py.....	7
5.8. model.py.....	7
6. Particionado.....	8
7. Linaje.....	9
8. API + Dashboard.....	10
8.1. API.....	10
8.2. Dashboard.....	10
9. Tests unitarios.....	11
10. Resultados Modelo Base.....	12

Hecho por: Gabriel Guzmán Puras Ruiz, Alvaro Ruiz Vallejo, Pablo Ruz Muñoz

Profesor: Jose Miguel Ruiz Guevara

1. EDA - Análisis Exploratorio de Datos

Primeramente, se han de analizar los datos utilizados en el trabajo para poder entender cual es el objetivo de nuestro modelo. Además, esto puede ayudar a detectar campos no necesarios para el entrenamiento del modelo.

1.1. Bases de datos: NASA Turbofan (CMAPSS)

La base de datos finalmente escogida es el dataset de la NASA enfocado a la predicción de fallo de motores. La idea principal es poder calcular el RUL (Remaining Useful Life). Para esto se nos dan 3 datasets con diferentes valores de RUL medio:

- **df_train:** Los datasets de entrenamiento son muy buenos y fuertes para el entrenamiento del modelo. Cada uno de los motores registrados en los datasets de entrenamiento acaban fallando. En la realidad, no existen prácticamente datos en los que acaben llegando al fallo el motor, ahí es donde entra **CMAPSS (Commercial Modular Aero-Propulsion System Simulation)**.
- **df_test:** Los datasets de testeo son mucho más variados, ya que su propósito es poder probar el modelo. En este dataset algunos motores acaban fallando y otros no, perfecto para poder probar el modelo.
- **RUL_df:** Este documento incluye los valores reales de RUL correspondiente al df_test. Se utilizará para el testeo del modelo, para poder ver los valores correctos y compararlos con las predicciones del modelo

Además existen 4 subconjuntos por cada conjunto de datos:

Conjunto de Datos	Trayectorias (Entrenamiento/Prueba)	Condiciones Operativas	Modos de Fallo
FD001	100 / 100	UNA (Nivel del mar)	UNO (Degradación del HPC)
FD002	260 / 259	SEIS (6 entornos)	UNO (Degradación del HPC)
FD003	100 / 100	UNA (Nivel del mar)	DOS (Degradación del HPC y del Fan)
FD004	248 / 249	SEIS (6 entornos)	DOS (Degradación del HPC y del Fan)

2. Calidad de Datos

Para un buen entrenamiento del modelo, la calidad de los datos presentes es fundamental. En general, la calidad de los datos presentes en el dataset de la NASA son extraordinarios, por lo que el procedimiento de enriquecer estos datos ha sido menor.

2.1. Los Datos del Dataset

En primer lugar, para entender un dataset hay que entender la información que contienen estos. Por lo tanto, vamos a ver un desglose de los datos:

CAMPO	IMPORTANCIA	EXPLICACIÓN
unit_number	CLAVE PRIMARIA	Es el número identificador del motor
time_in_cycles	CLAVE PRIMARIA	Es el tiempo dado en ciclos del motor. Será útil para el cálculo del RUL en train
op_setting_1, op_setting_2 y op_setting_3	X	“op” hace referencia a “Operational Parameter” y son las condiciones operativas
sensores(1..21)	X	El dataset incluye un total de 21 sensores que miden temperaturas, presión, velocidad y otras medidas.
RUL	X	Es el dato a predecir. En el dataset de testeo viene dado en un archivo distinto, pero en el caso de train se calculará...

2.2. Datos innecesarios

En el dataset proporcionado hemos podido detectar sensores cuya importancia es irrelevante o casi inexistente. La lógica aplicada es la siguiente:

En los datasets de entrenamiento, todos los motores fallan. Por lo que si el valor de un sensor no varía en ningún ciclo o en diferentes motores, o sólo varía una pizca, este dato es innecesario para la detección del RUL, ya que no tiene un impacto directo en el final que acaba teniendo el motor: el fallo.

Con esta premisa, hemos decidido eliminar aquellos datos cuya variación no pasa las 0.01 variaciones estándar.

3. Feature Engineering

A nivel de ingeniería de datos, no vamos a necesitar gran cosa, ya que prácticamente todos los datos necesarios vienen dados por el dataset. Únicamente, al dataset de entrenamiento le falta un dato primordial para poder entrenar el modelo: el **RUL**

3.1. Cálculo del RUL en entrenamiento

El cálculo del RUL en el dataset de entrenamiento es muy simple, para ello cabe recordar lo siguiente:

“Las trayectorias de entrenamiento se ejecutan hasta el fallo”

Por lo tanto, si estas trayectorias se ejecutan hasta el fallo, en el último ciclo de cada motor el RUL será 0. Ahora bien, para hacerlo de forma progresiva necesitamos una pequeña fórmula que defina esto correctamente:

$$\text{RUL}_{\text{train}} = \max(\text{time_in_cycles}) - \text{current_cycle}$$

De esta manera, el RUL acaba siendo el tiempo en ciclos, pero a la inversa y acabado en 0. Aunque no tiene porque ser 100% preciso, es lo mejor que podemos conseguir. El RUL no tiene porque bajar cada ciclo en 1, dependiendo del tratamiento al motor, puede incluso quedarse equilibrada.

4. Pipeline de datos

A continuación, se realiza una explicación del procedimiento que realiza el pipeline de datos para la ingesta, tratado y almacenamiento de datos.

4.1. Ingesta

La ingesta de datos define la manera y la cantidad de datos que entran en el pipeline. En nuestro proyecto la ingesta de datos se hace en **batch**, es decir, a montones.

Esta decisión viene dada por nuestra idea en el producto final. Queremos un modelo que pueda, posteriormente a un viaje, recoger las mediciones de los sensores de un motor y utilizarlo para calcular el RUL restante después del viaje. Además, la idea es poder prevenir una necesidad de mantenimiento en un motor, si en medio de un viaje el RUL llegase a 0, ya no se podría hacer nada, por lo que un monitoreo constante es innecesario.

Además de esto, la ingesta de los datos se hace en el formato proporcionado por el dataset, en formato **.txt**.

4.2. Tratado

El tratamiento de los datos se hacen entre varias funciones en las que se ha de asignar el nombre de las columnas al dataset, limpiar los datos (como se ha explicado anteriormente) y guardarlos. Además esto se ha de hacer sobre todo el conjunto de datos, que están separados en diferentes archivos

Para un primer guardado, hemos decidido primeramente guardar estos datos limpios en formato CSV, ya que es un formato claro para visualizar y muy común en el mundo del manejo de base de datos.

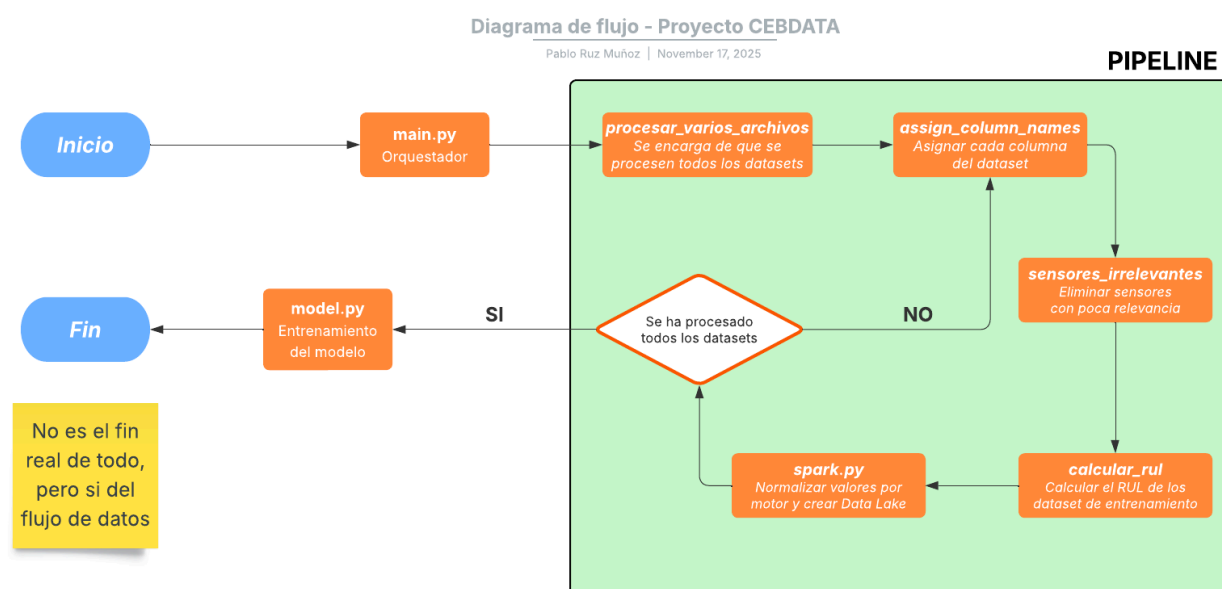
4.3. Almacenamiento

Por último, para el almacenamiento de los datos del pipeline de datos queremos construir un Data Lake de estos datos con ayuda de Apache Spark.

De esta manera, la idea final será almacenarlos en formato **.parquet**. El formato **.parquet** es un formato ampliamente utilizado en big data que presenta una estructura columnar y en binario. Estas dos características hacen que su rendimiento para el entrenamiento de modelos de IA sea efectivo y rápido.

5. Diagrama de flujo

A continuación se presentará un diagrama de flujo para entender todo el procedimiento por el que pasan los datos, desde la ingesta hasta la creación del Data Lake. Para una mejor visualización, en la siguiente imagen podrás ver un esquema referente al diagrama de flujo, el cual es accesible también haciendo click en la misma imagen (necesario iniciar sesión):



5.1. main.py

Este es el programa principal que orquesta todo lo que se realiza desde el pipeline de datos hasta la puesta en marcha del dashboard. Es un programa donde la estructura más básica del proyecto se puede entender.

En esta parte del programa, también se indicarán condiciones previas para ejecutar el programa, por ejemplo: el uso de pyspark. Apache Spark tiene muy mala compatibilidad con Windows, por lo que requiere de un sistema o subsistema Linux para tener el resultado deseado. Por este motivo, se ha añadido la posibilidad de no usar spark, aunque la calidad de la base de datos pueda ser algo peor y los ficheros parquet no se generaron

5.2. PIPELINE

Dentro del pipeline podemos encontrar todo el proceso por el que pasan los datos. Concretamente hace referencia a al programa pipeline.py

5.3. procesar_varios_archivos

Esta función se encarga de recoger los conjuntos de datos, que como se ha mencionado anteriormente vienen en distintos ficheros, y procesarlos uno a uno hasta que todos han sido procesados. Básicamente, itera por cada fichero y consigue devolver los archivos en varios CSV y en varios dataset pandas.

5.4. assign_column_names

Esta función se encarga de asignar el nombre de cada columna en el dataset. En el dataset original, que como se ha comentado previamente viene dado en .txt, no están el nombre de las columnas.

Por lo tanto, hay que indicarlo teniendo en cuenta el paper de la Nasa, siguiendo el mismo orden de los sensores y teniendo en cuenta que los primeros 5 campos no son sensores, sino campos identificadores y las condiciones del entorno.

5.5. sensores_irrelevantes

Como el nombre indica, la utilidad de esta función se basa en identificar los sensores que son irrelevantes y eliminarlos. La estrategia para la eliminación se ha comentado previamente, todo sensor cuyo variación no superé las 0.01 variaciones estándar

Este cálculo se produce sobre cada dataset, lo cual genera un problema que aún persiste: en cada dataset se eliminan campos distintos. En los datasets cuyas condiciones de entorno varían mucho se han eliminado menos valores que en los que el entorno no varía, lo cual sugiere que hay sensores que resultan más relevantes en según qué entornos.

5.6. calcular_rul

Función sencilla que se encarga de calcular el RUL (Remaining Useful Life) en el dataset de entrenamiento. Se ha comentado anteriormente también que este RUL se calcula teniendo en cuenta que acaba en 0 y usando el último ciclo de cada motor restado al ciclo actual

5.7. spark.py

Este módulo se encarga de todo lo correspondiente a spark. Se encarga tanto del arranque de spark como la construcción del Data Lake. Hemos decidido hacer este módulo como algo más "opcional". Por un motivo simple, Apache Spark es únicamente compatible con Linux, con Windows genera muchos problemas.

5.8. model.py

En este módulo se realiza principalmente el entrenamiento del modelo. También previamente se realizan algunos ajustes, en nuestro caso por ejemplo hemos normalizado los valores de los sensores, ya que ayuda a la comprensión del modelo y a evitar sesgos

También se generan métricas para poder reajustar el modelo y el dataset según los resultados presentados. El modelo base es de regresión, pero queremos realizar una red neuronal

6. Particionado

Los datos están particionados en entrenamiento y test de forma original. El dataset de test no se utilizará para entrenar el modelo, si no para testearlo. Por lo tanto el dataset de testeo no tiene mucho más misterio.

En cambio el dataset de entrenamiento se ha de particionar en 2 para el entrenamiento del modelo, uno de entrenamiento y otro de validación. El dataset de validación se utiliza en el entrenamiento del modelo para evaluar el rendimiento durante el entrenamiento.

Generalmente, se suelen validar estas métricas entre epochs. Esto va a permitir valorar un posible overfitting o un entrenamiento poco fructífero.

El particionado del dataset se puede hacer de varias maneras, las cuales estamos estudiando para ver cual puede presentar un mejor resultado:

- **En una proporción de 85/15 (o tal vez en diferente proporción):** 85% de datos de entrenamiento y 15% de validación. Básicamente se cogerían un 15% de los datos de entrenamiento y se aplicarían como datos de validación, eso si, tienen que tener variedad. Es decir, tanto los datos de entrenamiento como de validación han de contener varios ejemplos de condiciones operativas distintas.
- **Utilizar los dataset de testeo:** Realmente, al tener tantos datos de testeo podríamos llegar a sacrificar algunos para crear un dataset de validación propio con varios motores de los diferentes datasets de testeo. Los datos de testeo son obviamente necesarios, pero la cantidad de datos presentes son abundantes y pueden no ser necesarios tantos.

El objetivo de un buen particionado es tratar que los datos de entrenamiento, validación o testeo tengan proporciones adecuadas para mejorar el rendimiento del modelo y poder testearlo sin problemas. Con este objetivo en mente, buscaremos el mejor particionado posible para nuestro caso, intentando asegurar que los datos de entrenamiento sean completos y que hayan suficientes datos tanto de validación, como de testeo.

7. Linaje

El linaje de los datos de nuestro proyecto es claro y se ha ido explicando en el transcurso de este documento. A continuación, se podrá ver el proceso de forma más clara y explicativa:

1. **Origen de los datos:** Los datos nos llegan en formato .txt. Estos están instalados de forma local y guardados en data/raw_data. Los datos que tenemos al principio ya parten de una base sólida, pero necesitan algunas transformaciones y mejoras antes de llegar al modelo
2. **Transformaciones:** Sobre estos datos se realizan varias transformaciones de cara a mejorar los dataset y su entendimiento:
 - **Asignar columnas:** Los datos dados en el .txt original vienen sin los nombres de los campos, por lo que estos se tienen que asignar de forma manual
 - **Eliminar sensores:** Hay sensores cuya relevancia es inexistente según el entorno, y se acaban extirpando del dataset
 - **Obtener RUL:** En el dataset de entrenamiento no tenemos el valor que queremos predecir, es decir el RUL. Por lo que hemos hecho un cálculo para poder predecirlo en un futuro.
 - **Normalización:** Esta transformación se efectúa antes del entrenamiento del modelo y sirve para que el modelo pueda comprender mucho mejor los valores de los sensores, reduciendo el rango que comprende en números menores y más entendible para el modelo. Esto también ayuda a reducir el impacto de pequeños outliers y reducir sesgos
3. **Guardados:** Los datos que han sufrido ya las transformaciones (menos la normalización) se guardan en **output**. Los datasets se presentan en dos guardados:
 - **(nombre-dataset)_filtrado.csv:** Es el guardado principal que hemos querido utilizar. Los archivos CSV son fáciles de utilizar y efectivos para la lectura.
 - **(nombre-dataset)_parquet:** Es el guardado en forma de Data Lake. Este guardado solo se efectúa si al principio del programa se indica el uso de pyspark, el cual únicamente se recomienda para entornos Linux o subsistemas usando por ejemplo WSL2.
4. **Destino:** El destino final de los datos es un dataset de pandas que pueda ser procesado correctamente por el modelo, a través de la partición del dataset en 2 particiones: entrenamiento y validación. Todo esto para poder, posteriormente, testearlo con el dataset de testing

8. API + Dashboard

En esta fase 2, la base de la API y el dashboard también están construidas. Se puede ubicar su código dentro del trabajo y ver las tecnologías aplicadas, las cuales corresponden a la arquitectura de alto nivel definida en la fase 1.

8.1. API

La tecnología empleada para la construcción de una API REST es **FastAPI**. Esta es una librería de python sencilla para la construcción de aplicaciones. Como se ha mencionado anteriormente, esta parte aún se encuentra en una fase previa, ya que el modelo que tenemos aún no es el definitivo.

Por el momento, se puede encontrar en el programa de la API el arranque y puesta en marcha de la misma y el monitoreo de la misma.

En un futuro, se añadirán la petición POST necesaria para llamar al modelo y que realice una predicción y, seguramente, un GET para obtener la respuesta. Como digo, esto aún no está completamente mirado, lo exploraremos en profundidad en la fase 3.

8.2. Dashboard

La tecnología empleada para la construcción del dashboard ha sido **streamlit**. Al igual que con FastAPI, streamlit es una librería de python sencilla para la construcción de una interfaz de usuario. En este caso hemos añadido una estructura sencilla de los títulos y la estructuración que, más o menos, tenemos pensada para la interfaz.

Primeramente, se mostrará un resumen con x cantidad de motores con el RUL más bajo, ya que estos son los prioritarios a nivel de mantenimiento. Posteriormente, queremos también hacer un buscador por motor para poder ver analíticas más avanzadas de un motor en concreto.

En un futuro, se implementarán los gráficos y lo necesario para conseguir nuestro objetivo. Por el momento, al tener únicamente un modelo base, hemos decidido dejarlo en standby hasta que el modelo sea definitivo y así poder realizar mejores pruebas de la interfaz

9. Tests unitarios

Los tests unitarios que hemos implementado en nuestro código verifican el correcto funcionamiento de las funciones. Los tests establecen unas normas referentes al resultado y a la ejecución de la función que se han de seguir. Estos tests se ubican en **src/tests**

En nuestro caso, estamos tratando de implementar un test unitario por cada función del código y, a su vez, un módulo de test por cada módulo de código.

Por indicar algún ejemplo:

```
def test_assign_column_names_ok():
    # Simular dataset crudo con 26 columnas (5 + 21 sensores)
    data = [[i for i in range(26)] for _ in range(3)]
    df_raw = pd.DataFrame(data)

    df_named = assign_column_names(df_raw)

    # Validaciones
    assert isinstance(df_named, pd.DataFrame)
    assert len(df_named.columns) == 26
    assert "unit_number" in df_named.columns
    assert "T2" in df_named.columns
    assert "W32" in df_named.columns
    assert df_named.columns[0] == "unit_number"
    assert df_named.columns[-1] == "W32"
```

En este ejemplo se comprueba el funcionamiento de la función `assing_column_names`. Se crea un dataset de simulación con las 26 columnas del dataset original y 3 filas. Esto se convierte en un dataframe de pandas y se pasa a la función original, cuyo resultado se guarda en una variable a la que se le hará comprobaciones. En este caso, se hacen la siguiente comprobaciones (por orden):

1. ¿Existe la variable y es un dataframe de pandas?
2. ¿La longitud se ha mantenido igual?
3. ¿Se puede encontrar la columna "unit_number"?
4. ¿Se puede encontrar la columna "T2"?
5. ¿Se puede encontrar la columna "W32"?
6. La primera columna es "unit_number"?
7. La última columna es "W32"?

Si la respuesta acaba siendo afirmativa a todas estas preguntas, entonces el test se ha pasado satisfactoriamente. De lo contrario, saltará un `AssertionError`

Para probar los tests, se utiliza el comando **pytest** desde la raíz del proyecto.

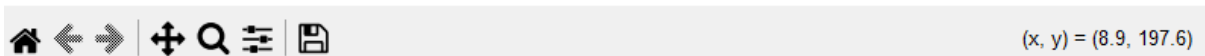
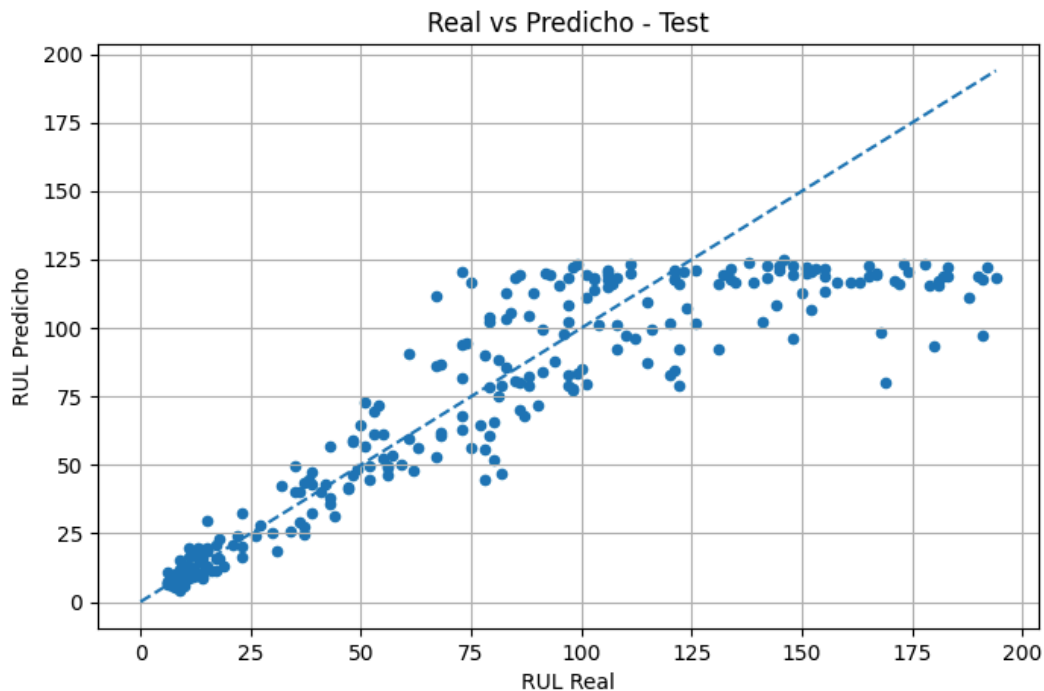
10. Resultados Modelo Base

A continuación se muestran un resumen de los resultados del modelo base:

MSE: 680.29, RMSE: 26.08, R2: 0.765, NASA_Score: 7113.380

	Motor	RUL_real	RUL_predicho
0	1	18.0	15.949381
1	2	79.0	102.269554
2	3	106.0	120.883926
3	4	110.0	97.225990
4	5	15.0	19.660128

Figure 1



Podemos observar en el gráfico que las predicciones del RUL se desvían un poco más de la realidad en valores altos.

También se observa que una de las medidas que hemos tomado es captar el RUL en 125, de esta manera el modelo no es capaz de predecir por encima. Esta idea se basa en el hecho de que en cantidades de RUL tan altas, los valores de los sensores no varían casi, y eso complicaría el entendimiento del modelo